

Tests Boîte Noire

Lectures: Ron Patton chap 5, Daniel Galin chap 9.5.1

Avec du matériel de: R. Binder, Testing Object-Oriented Systems

Tests boîte-noire ?

- ◆ Tests sans la connaissance du code sous-jacent



- Tests basés sur la *spécification*

Tests boîte-noire ?

- Avantages?
- Dés-avantages?

Tests boîte-noire ?

- Approches:
 - Divination d'erreurs (Error guessing)
 - Partitionnement en Classe d'équivalences
 - Analyse de Valeurs aux Bornes
 - Partitionnement en Catégories
 - Tables de Décision
 - Graphes Causes-Effets
 - Tests dérivés des exigences (ex: cas d'utilisations)

Divination d'erreurs (1)

- ◆ Approche ad-hoc basée sur l'expérience
- ◆ Approche:
 1. Faire une liste d'erreurs possibles ou situations conduisant à des erreurs
 - modèle d'erreurs
 2. Développer des cas de tests pour couvrir le modèle d'erreurs
- Développez et maintenez vos modèles d'erreurs

Divination d'erreurs (2)

- ◆ Exemple fonction de tri de tableau
- ◆ Modèle d'erreur:
 - tableau vide
 - tableau trié
 - tableau trié à l'envers
 - grand tableau non trié
 - ...
- ◆ Générer des cas de tests pour ces situations

Partitionnement en Classes d'équivalences

- ◆ Partition du domaine de données en classes d'équivalences selon la spécification
- ◆ Idéalement les CEs devraient être
 - telles que chaque donnée est dans une classe
 - disjointes
 - les éléments d'une même classe mis en correspondance avec leur résultats de façon similaire
- ◆ En pratique il est difficile de déterminer des CEs *parfaites*
 - usage d'heuristiques

Heuristiques pour Identification de Classes d'équivalences

Pour chaque donnée:

1. si spécifie un intervalle de valeurs valides, définir
 - 1 CE valide (dans l'intervalle)
 - 2 CE invalides (une à chaque bout de l'intervalle)
2. si spécifie un nombre (N) de valeurs valides, définir
 - 1 CE valide
 - 2 CE invalides (aucune et plus de N)
3. si spécifie un ensemble de valeurs valides, définir
 - 1 CE valide (dans l'ensemble)
 - 1 CE invalide (en dehors de l'ensemble)

Heuristiques pour Identification de Classes d'équivalences

Pour chaque donnée:

4. si spécifie une situation *doit être*, définir
 - 1 CE valide (satisfaisant le *doit être*)
 - 1 CE invalide (ne satisfaisant pas le *doit être*)
5. si il y a une raison de croire que les éléments d'une CE ne sont pas traités de façon identiques par le programme
 - subdiviser la CE en plus petites CEs.

Heuristiques pour Identification de Classes d'équivalences

- Considérez la création de classes d'équivalences pour les conditions suivantes: défaut, vide, blanc, nulle, zéro, ou aucune.

Partitionnement en Classes d'équivalences - Exemple (1)

◆ Spécification

- données trois entiers (côtés du triangle: a, b, c)
 - chaque côté doit être un nombre positif inférieur ou égal à 20.
- résultat type du triangle:
 - Équilatéral si $a = b = c$
 - Isocèles si 2 paires de côté sont égaux
 - Scalène si aucun côté n'est égal à l'autre ou
 - PasUnTriangle si $a \geq b + c$, $b \geq a + c$, ou $c \geq a + b$

Partitionnement en Classes d'équivalences - Exemple (2)

Selon l'heuristique #5

Condition de donnée	CE Valide	CE Invalide
Côtés (a, b, c)	tous > 0 et ≤ 20 (1)	a >20 (2) b >20 (3) c >20 (4) a ≤ 0 (5) b ≤ 0 (6) c ≤ 0 (7)

Partitionnement en Classes d'équivalences - Exemple (3)

CE #1 trop large peut-être subdivisée (heuristique #5)

- ◆ Basé sur le traitement de données
 - 1.1. a, b, c tel que le triangle est équilatéral
 - 1.2. a, b, c tel que le triangle est isocèle
 - 1.3. a, b, c tel que le triangle est scalène
 - 1.4. a, b, c tel que ce n'est pas un triangle

Partitionnement en Classes d'équivalences - Exemple (3)

CE #1 trop large peut-être subdivisée (heuristique #5)

◆ Basé sur les données

1.5. $a = b = c$

1.6. $a = b, a \neq c$

1.7. $a = c, a \neq b$

1.8. $b = c, a \neq b$

1.9. $a \neq b, a \neq c, b \neq c$

Partitionnement en Classes d'équivalences - Exemple (3)

CE #1 trop large peut-être subdivisée (heuristique #5)

- ◆ Basé sur la propriété de triangle

- 1.10. a, b, c tel que $a \geq b + c$

- 1.11. a, b, c tel que $b \geq a + c$

- 1.12. a, b, c tel que $c \geq a + b$

Partitionnement en Classes d'équivalences – Identification de cas de tests

Approche de sélection de Myer

1. Jusqu'à ce que toutes les CEs valides soient couvertes par des cas de tests,
 - écrire un nouveau cas de test couvrant le plus de CEs valides non couvertes possible
2. Jusqu'à ce que toutes les CEs invalides soient couvertes par des cas de tests,
 - écrire un nouveau cas de test couvrant *une et seulement une* des CEs invalides non couvertes

Partitionnement en Classes d'équivalences - Exemple (4)

◆ Cas de Tests pour triangle

EC	a	b	c	Résultat attendu
1.1, 1.5	12	12	12	Equilatéral
1.2, 1.6	6	6	7	Isosceles
1.3, 1.9	9	3	7	Scalene
1.4, 1.10	8	2	3	Pas un triangle
2	30	15	6	
3	1	21	19	
4	7	14	22	
5	-5	10	11	
6	12	-10	10	
7	8	5	-1	

Analyse des Valeurs aux Bornes (AVB)

- ◆ Erreurs ont tendance à survenir au tours des valeurs extrêmes (*bornes*)
- ◆ AVB améliore le Partitionnement en Classes d'équivalences en
 - sélectionnant les éléments juste à et autour des bornes de chacune des CEs
 - dérivant des cas de tests en considérant des CEs des résultats également

Conditions de bornes

- Situation à la bordure des limites opérationnelles envisagées
- Types de données ayant des conditions de bornes
 - Numérique, Caractère, Position, Quantité, Vitesse, Location, Taille
- Caractéristiques de conditions de bornes
 - Premier/Dernier, Début/Fin, Minimum/Maximum, Au-dessus/Au-dessous, Vide/Plein, Plus-lent/Plus-rapide, Plus-grand/Plus-petit, Plus-proche-de/Plus-loin-de, Plus-court/Plus-long, Plus-tôt/Plus-tard, Plus-haut/Plus-bas

Analyse des Valeurs aux Bornes - Directives

- ◆ Pour chaque condition de borne
 - ajouter la valeur de borne correspondante dans au moins un cas de test valide
 - ajouter la valeur juste au delà (ou en deçà) de la valeur de borne correspondante dans au moins un cas de test invalide.
- ◆ Appliquer les mêmes directives pour les résultats
 - inclure des cas de tests avec des données, tels que des résultats aux bornes sont produits

Analyse des Valeurs aux Bornes

- Conditions de bornes secondaires
 - internes (non définis dans la spécification)
 - exemples:
 - Puissances-de-deux: bit, nibble, byte, mot, kilo, mega, giga, tera
 - table ASCII

Analyse des Valeurs aux Bornes

	Caractère	Code ASCII	
	/	47	
lower bound	0	48	Si l'entrée consiste en des nombres (conversion selon la table ascii), '/' et ':' sont de possibles valeurs de bornes.
	1	49	
	2	50	
	3	51	
	4	52	
	5	53	
	6	54	
	7	55	
	8	56	
upper bound	9	57	
	:	58	
	A	65	
	a	97	

Partitionnement en Classes d'équivalences

- Avantages

- Nécessite peu de cas de tests
- Probabilité de détection de défauts avec les cas de tests sélectionnés est supérieure à celle obtenue par le même nombre de cas de tests choisis au hasard

- Limitations

- Spécification ne définit pas toujours les résultats attendus pour les cas de tests *invalides*
- Langages fortement typés éliminent le besoin de la considération de certaines données invalides
- Approche de sélection de Myer est faible lorsque les variables de données *ne sont pas indépendantes*

Partitionnement en Classe d'équivalences - Nextdate (1)

- ◆ Données: *mois, jour, an* représentant une *date*
 - $1 \leq \text{mois} \leq 12$
 - $1 \leq \text{jour} \leq 31$
 - $1812 \leq \text{an} \leq 2022$
- ◆ Résultat: date du jour suivant la date donnée
 - Doit considérer les années bissextiles
 - Année bissextile si divisible par 4 et pas siècle
 - Année siècle bissextile si multiple de 400

Partitionnement en Classes d'équivalences - Nextdate (2)

◆ Classes d'équivalences

Condition de donnée	CEs Valides	CEs Invalides
mois	$1 \leq \text{mois} \leq 12$	mois < 1 mois > 12
jour	$1 \leq \text{jour} \leq 31$	jour < 1 jour > 31
an	$1812 \leq \text{an} \leq 2022$	an < 1812 An > 2022

- Raffinement possible selon la façon dont les données sont traitées
 - Jour incrémenté de 1 à moins que dernier jour d'un mois
 - Dépend du mois (30, 31, Février)
 - A la fin du mois, prochain jour est 1 et mois est incrémenté
 - A la fin d'une année, jour et mois remis à 1, et an incrementé
 - Problème d'années bissextiles

Partitionnement en Classes d'équivalences - Nextdate (3)

◆ Décomposition des CEs valides

Condition de données	CEs Valides
mois	30 jours (1)
	31 jours (2)
	Février (3)
jour	$1 \leq \text{jour} \leq 28$ (4)
	jour = 29 (5)
	jour = 30 (6)
	jour = 31 (7)
an	an = 1900 (8)
	$1812 \leq \text{an} \leq 2022$ et an $\neq$ 1900 et an mod 4 = 0 (9)
	$1812 \leq \text{an} \leq 2022$ et an mod 4 $\neq$ 0 (10)

Partitionnement en Classes d'équivalences - Nextdate (4)

- ◆ Cas de Tests obtenus par la sélection de Myer

ID	Ecs	month	day	year	output
1	1, 4, 8	6	14	1900	15/06/1900
2	2, 5, 9	1	29	1988	30/01/1988
3	3, 6, 10	2	30	2001	Error
4	2, 7, 8	7	31	1900	01/08/1900

Meilleure couverture obtenue en sélectionnant les cas de tests selon le produit Cartésien des CEs
→ 36 cas de tests.

Partitionnement en Classes d'équivalences - Nextdate (4)

◆ Cas de Tests par produit Cartésien

ID	CE	mois	jour	an	résultat
1	1, 4, 8	6	14	1900	15/06/1900
2	1, 4, 9	6	14	1912	15/06/1912
3	1, 4, 10	6	14	1913	15/06/1913
4	1, 5, 8	6	29	1900	30/06/1900
5	1, 5, 9	6	29	1912	30/06/1912
6	1, 5, 10	6	29	1913	30/06/1913
7	1, 6, 8	6	30	1900	01/07/1900
8	1, 6, 9	6	30	1912	01/07/1912
9	1, 6, 10	6	30	1913	01/07/1913
10	1, 7, 8	6	31	1900	Erreur
11	1, 7, 9	6	31	1912	Erreur
12	1, 7, 10	6	31	1913	Erreur

Partitionnement en Classes d'équivalences

- ◆ *L'approche force-brute* de définition de cas de test pour chaque combinaison de CEs
 - donne une bonne couverture, mais
 - non pratique lorsque le nombre d'entrées et classes associé est grand

Partitionnement en Catégories

- Méthodologie systématique basée spécification, qui utilise une spécification fonctionnelle informelle pour produire une spécification de test formelle
- Consiste en:
 - identification de *catégories*,
 - partitionnement des catégories en *choix*,
 - création de *cadres de tests* par la sélection de choix et création de *cas de tests*

Partitionnement en Catégories - Étapes

- Décomposez la spécification fonctionnelle en unités fonctionnelles
 - pour tester séparément
- Examinez chacune des unités fonctionnelles
 - Identifiez les paramètres
 - Données explicites de l'unité fonctionnelle
 - Conditions environnementales
 - Caractéristiques de l'état du système

Partitionnement en Catégories - Étapes

- Trouver des catégories pour chaque paramètre et condition environnementale
 - Propriété majeure ou caractéristique d'un paramètre/condition
- Trouver des choix pour chaque catégorie
 - Partition distincte d'une catégorie

Partitionnement en Catégories - Étapes

- Déterminer des contraintes sur les choix
 - comment les choix interagissent, comment un choix peut affecter un autre, et quelles sont les restrictions particulières susceptibles d'affecter les choix
- Créer des cadres de tests (test frame)
 - ensemble de choix une pour chaque catégorie
- Créer des cas de tests
 - cadre de test avec des valeurs spécifiques pour chacun des choix

Partitionnement en Catégories -

Exemple

Command: find

Syntax: find <pattern> <file>

Function: The find command is used to locate one or more instance of a given pattern in a text file. All lines in the file that contain the pattern are written to standard output. A line containing the pattern is written only once, regardless of the number of times the pattern occurs in it.

The pattern is any sequence of characters whose length does not exceed the maximum length of a line in the file .To include a blank in the pattern, the entire pattern must be enclosed in quotes (“”).To include quotation mark in the pattern, two quotes in a row (“ “) must be used.

Parameter: Pattern

Pattern size:

- empty
- single character
- many character
- longer than any line in the file

Quoting:

- pattern is quoted
- pattern is not quoted
- pattern is improperly quoted

Embedded blanks:

- no embedded blank
- one embedded blank
- several embedded blanks

Paramètre

Catégorie

Choix



Embedded quotes:

- no embedded quotes
- one embedded quotes
- several embedded quotes

Parameter: File

File name:

- good file name
- no file with this name

Environments:

Number of occurrence of pattern in file:

- none
- exactly one
- more than one

Pattern occurrences on target line:

- one
- more than one

Exemple de Cadre de Test :

Pattern size : many characters

Quoting : pattern is quoted

Embedded blanks : several embedded blanks

Embedded quotes : no embedded quote

File name : good file name

Number of occurrence of pattern in file : none

Pattern occurrence on target line : one

Partitionnement en Catégories - Contraintes

- Permet d'éliminer des cadres contradictoires/infaisables
- Propriétés
 - assignés aux choix et vérifiés par d'autres choix
 - Format: **[property A, B, ...]**
 - A et B sont des noms de propriétés
 - Ex : [property Empty]

Partitionnement en Catégories - Contraintes

- Expression sélecteur
 - conjonction de propriétés assignés à d'autres choix
 - **[if A]**
 - Ex : [if Empty]

Parameters:

Pattern size:

empty	[property Empty]
single character	[property NonEmpty]
many character	[property NonEmpty]
longer than any line in the file	[property NonEmpty]

Quoting:

pattern is quoted	[property quoted]
pattern is not quoted	[if NonEmpty]
pattern is improperly quoted	[if NonEmpty]

Embedded blanks:

no embedded blank	[if NonEmpty]
one embedded blank	[if NonEmpty and Quoted]
several embedded blanks	[if NonEmpty and Quoted]

Embedded quotes:

no embedded quotes	[if NonEmpty]
one embedded quotes	[if NonEmpty]
several embedded quotes	[if NonEmpty]

File name:

good file name
no file with this name

Environments:

Number of occurrence of pattern in file:

none	[if NonEmpty]
exactly one	[if NonEmpty] [property Match]
more than one	[if NonEmpty] [property Match]

Pattern occurrences on target line:

one	[if Match]
more than one	[if Match]

Partitionnement en Catégories – annotation d'erreur

- Pour limiter les cadres redondantes lorsqu'un paramètre ou condition cause une erreur
- Annotation **[error]**
 - ajouté à un choix produisant une erreur
indépendamment de la valeur des autres choix
 - choix annoté n'est pas combiné avec les choix des autres catégories pour créer des cadres de tests
 - cadre de test contient uniquement le choix annoté

Parameters:

Pattern size:

empty	[property Empty]
single character	[property NonEmpty]
many character	[property NonEmpty]
longer than any line in the file	[error]

Quoting:

pattern is quoted	[property quoted]
pattern is not quoted	[if NonEmpty]
pattern is improperly quoted	[error]

Embedded blanks:

no embedded blank	[if NonEmpty]
one embedded blank	[if NonEmpty and Quoted]
several embedded blanks	[if NonEmpty and Quoted]

Embedded quotes:

no embedded quotes	[if NonEmpty]
one embedded quotes	[if NonEmpty]
several embedded quotes	[if NonEmpty]

File name:

good file name	
no file with this name	[error]

Environments:

Number of occurrence of pattern in file:

none	[if NonEmpty]
exactly one	[if NonEmpty] [property Match]
more than one	[if NonEmpty] [property Match]

Pattern occurrences on target line:

one	[if Match]
more than one	[if Match]

Partitionnement en Catégories – annotation single

- Pour décrire les conditions spéciales, inhabituelles ou redondants qui n'ont pas à être combiné avec tous les choix possibles.
- Annotation [**single**]
 - jugement par le testeur que le choix marqué peut-être adéquatement testé avec un seul cas de test
 - choix annoté n'est pas combiné avec des choix dans les autres catégories pour créer des cadres de tests
 - cadre de test contiens seulement le choix annoté

Embedded quotes:

no embedded quotes	[if NonEmpty]
one embedded quotes	[if NonEmpty]
several embedded quotes	[if NonEmpty] [single]

File name:

good file name	
no file with this name	[error]

Environments:

Number of occurrence of pattern in file:

none	[if NonEmpty] [single]
exactly one	[if NonEmpty] [property Match]
more than one	[if NonEmpty] [property Match]

Pattern occurrences on target line:

one	[if Match] [single]
more than one	[if Match]

Tables de Décisions

- ◆ Idéales pour situations où:
 - une combinaison d'actions est sélectionnée sous un ensemble de conditions variables
 - les conditions dépendent des variables de données
 - la réponse produite ne dépend pas de l'ordre d'assignation ou évaluation des données
 - la réponse produite ne dépend pas d'entrées/sorties précédentes

Format de Tables de Décisions

Conditions	Combinaison de conditions (variantes)
Actions	Actions sélectionnées

Développement de Tables de Décisions

1. Identifier variables et conditions de décisions
2. Identifier actions résultantes sélectionnées ou contrôlées
3. Identifier quelle action doit être produite en réponse à une combinaison d'actions particulière

Exemple

- Supposez les règles de renouvellement de police d'assurance suivantes
 - 0 réclamations, $\text{âge} \leq 25$: augmentation de \$50
 - 0 réclamations, $\text{âge} > 25$: augmentation de \$25
 - 1 réclamation, $\text{âge} \leq 25$: augmentation de \$100, envoi d'une lettre
 - 1 réclamation, $\text{âge} > 25$: augmentation de \$50
 - 2, 3 ou 4 réclamations, $\text{âge} \leq 25$: augmentation de \$400, envoi d'une lettre
 - 2, 3 ou 4 réclamations, $\text{âge} > 25$: augmentation de \$200, envoi d'une lettre
 - plus de 4 réclamations: suppression de la police

Tables de Décisions – Triangle

Exemple

- ◆ **Variables de Décision:** côtés a , b , c
- ◆ **Actions – déterminer que**
 - pas un triangle
 - triangle scalène
 - triangle isocèle
 - triangle équilatéral
 - erreur
 - variante est impossible
- ◆ **Conditions**
 - $a < b+c$
 - $b < a+c$
 - $c < a+b$
 - $a=b$
 - $a=c$
 - $b=c$
 - $a > 20$
 - $b > 20$
 - $c > 20$

Tables de Décisions – Triangle

Exemple

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
c1: $a < b + c$	N	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	-	-	-
c2: $b < a + c$	-	N	Y	Y	Y	Y	Y	Y	Y	Y	Y	-	-	-
c3: $c < a + b$	-	-	N	Y	Y	Y	Y	Y	Y	Y	Y	-	-	-
c4: $a = b$	-	-	-	Y	Y	Y	Y	N	N	N	N	-	-	-
c5: $a = c$	-	-	-	Y	Y	N	N	Y	Y	N	N	-	-	-
c6: $b = c$	-	-	-	Y	N	Y	N	Y	N	Y	N	-	-	-
c7: $a > 20$	N	N	N	N	N	N	N	N	N	N	N	Y	-	-
c8: $b > 20$	N	N	N	N	N	N	N	N	N	N	N	-	Y	-
c9: $c > 20$	N	N	N	N	N	N	N	N	N	N	N	-	-	Y
a1: not a triangle	X	X	X											
a2: Scalene											X			
a3: Isosceles							X		X	X				
a4: Equilateral				X										
a5: impossible					X	X		X						
a6: error												X	X	X

- **Attention** aux *inconsistances*

Tables de Décisions – Condition Don't Care

- Condition *Don't Care*
 - Peut-être *vraie* ou *fausse* sans changer l'action
 - Simplifie la table de décision
 - Correspond à différentes implémentation possibles:
 - données sont nécessaires mais n'ont pas d'effet pour la variante
 - données peuvent être omises mais n'ont aucun effet lorsque fournies
 - **Exclusion Type-Sûre**
 - plusieurs conditions définies pour une variable non-binaire qui ne peut contenir qu'une valeur à la fois

Tables de Décisions – Conditions Can't Happen & Don't know

Attention à ne pas confondre une condition *Don't Care* avec

- Hypothèse *Can't Happen* - reflète une hypothèse selon laquelle
 - des entrées sont mutuellement exclusifs,
 - des entrées ne peuvent-être produite par l'environnement, ou
 - l'implémentation est structurée telle que l'évaluation n'est pas possible
- Situation *Don't Know* – reflète un modèle incomplet
- Généralement l'indication d'un problème avec la spécification
 - des tests sont nécessaires pour ces cas non définis

Table de Décision – Stratégies de génération de tests

- ◆ Toutes-Variantes explicites
 - un cas de test par variante explicite
 - peut-être améliorée avec des valeurs bornes
- ◆ Toutes-Variantes
 - un cas de test par variante (implicite)
 - 2^n variantes pour n conditions
- ◆ Toutes-Vraies
 - un cas de test par variante avec résultat (action)
- ◆ Toutes-Fausses
 - un cas de test par variante sans résultat

Tables de Décisions – génération de tests exemple

◆ Toutes-Variantes explicites

Variante	a	b	c	Résultat attendu
1	4	1	2	Pas un Triangle
2	1	4	2	Pas un Triangle
3	1	2	4	Pas un Triangle
4	5	5	5	Equilatéral
5				Impossible
6				Impossible
7	2	2	3	Isocèle
8				Impossible
9	2	3	2	Isocèle
10	3	2	2	Isocèle
11	3	4	5	Scalène
12	21	6	12	Erreur
13	6	45	17	Erreur
14	12	12	24	Erreur

Tables de Décisions – Stratégies de génération de tests

- Chaque-Condition/Toutes-Conditions
 - Heuristique pour réduire le nombre de variantes
- Ensemble de variantes comprend
 - Pour chaque variable, une variante où celle-ci est mise à *vraie* une fois avec toutes les autres variables *fausses*, et
 - Une variante où toutes les variables sont *vraies* (*and* logic),
ou
 - Une variante où toutes les variables sont *fausses* (*or* logic)

Tables de Décisions – Chaque-Condition/Toutes-Conditions

$S = P \text{ or } Q \text{ or } R$

P	Q	R	S (action)
F	F	T	x
F	T	F	x
T	F	F	x
F	F	F	

Tables de Décisions – Chaque-Condition/Toutes-Conditions

$S = P \text{ and } Q \text{ and } R$

P	Q	R	S (action)
F	F	T	
F	T	F	
T	F	F	
T	T	T	x

Tables de Décisions – Chaque-Condition/Toutes-Conditions

$Z = (A \text{ and } B \text{ and } (\text{not } C)) \text{ or } (A \text{ and } D)$

Développez les variantes pour $(A \text{ and } B \text{ and } (\text{not } C))$ ainsi que pour $(A \text{ and } D)$

A	B	C	D	Z (action)
T	F	T	-	
F	T	T	-	
F	F	F	-	
T	T	F	-	x
T	-	-	F	
F	-	-	T	
T	-	-	T	x

- Des valeurs doivent être assignées aux variables

Don't Care

- de façon aléatoire ou
- par suspicion

Tables de Décisions – Chaque-Condition/Toutes-Conditions

$Z = (A \text{ and } B \text{ and } (\text{not } C)) \text{ or } (A \text{ and } D)$

A	B	C	D	Z (action)
T	F	T	T	
F	T	T	F	
F	F	F	F	
T	T	F	F	x
T	T	F	F	
F	T	T	T	
T	T	T	T	x

- Exemple d'ensemble de variantes utilisé pour la génération de la suite de tests

Tables de Décisions – Chaque-Condition/Toutes-Conditions

- Fournit une table compacte
 - nombre de tests augmente de façon linéaire avec le nombre de termes produits de la formule
- Suppose l'indépendance de l'évaluation des conditions ainsi que l'absence d'actions par défaut

Tables de Décisions – Déterminants de Diagramme de Décision Binaires

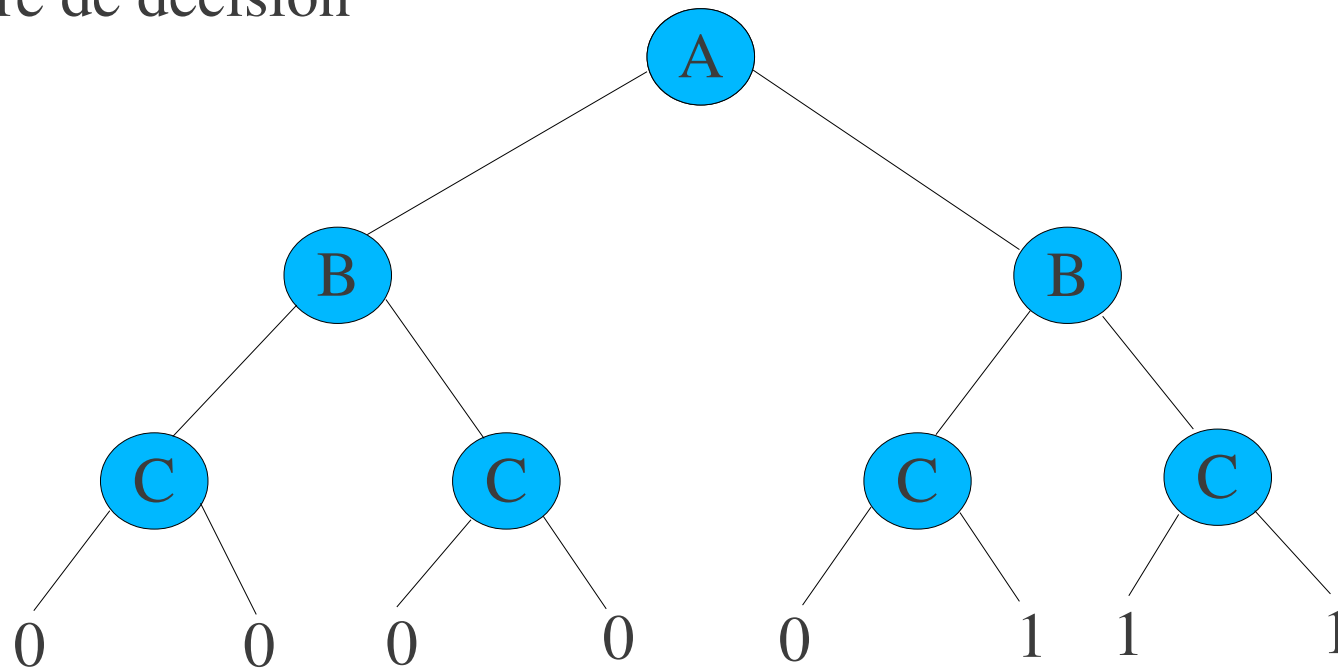
Diagramme de Décision Binaire (Binary Decision Diagram BDD) – représentation compacte d'une table de vérité

1. Construire une DDB à partir de la table de vérité
 - a) Créer un arbre de décision à partir de la table de vérité
 - Noeuds représentent des variables Booléennes
 - Branche de gauche toujours *fausse*
 - Branche de droite toujours *vraie*
 - Chaque branche représente la valeur résultante de la condition sur le chemin de la racine à la feuille

Tables de Décisions – Déterminants de Diagramme de Décision Binaires

Exemple: $Z = A \text{ and } (B \text{ or } C)$

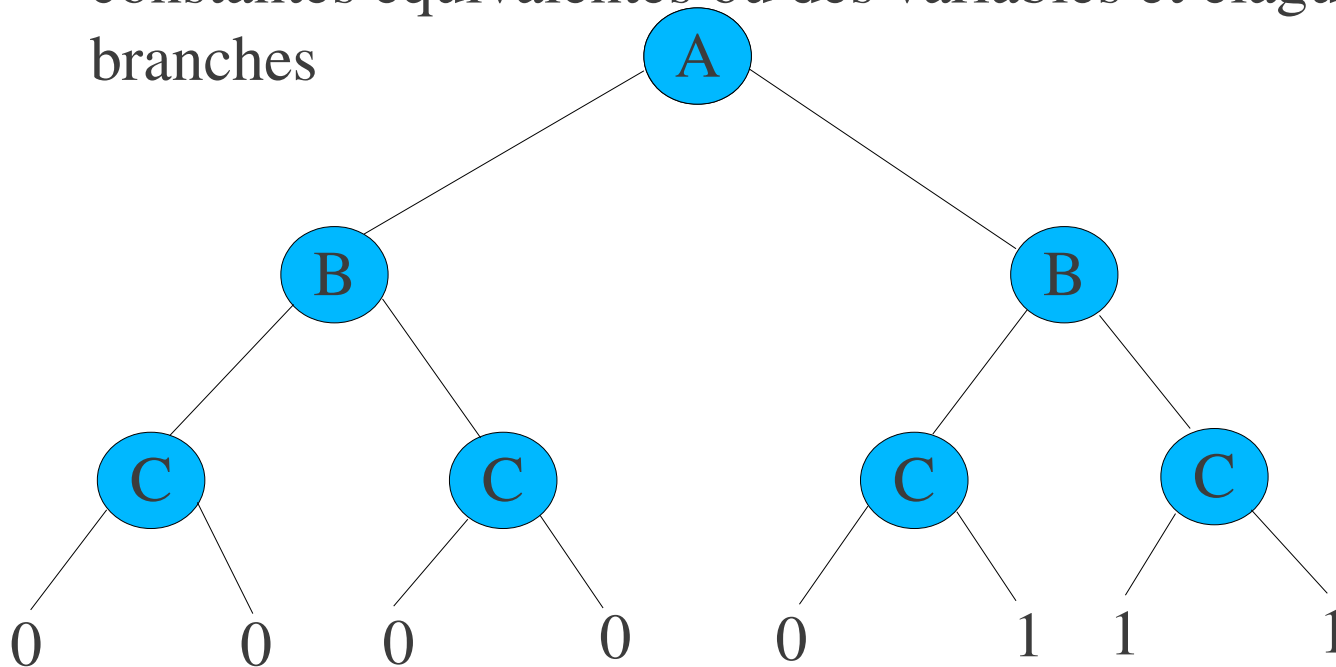
Arbre de décision



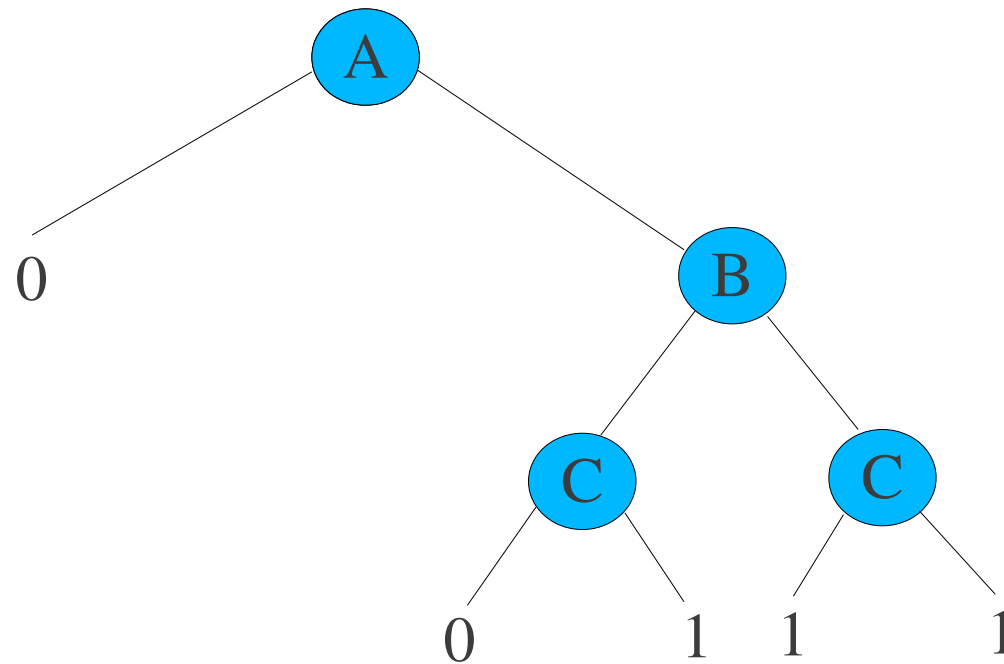
Tables de Décisions – Déterminants de Diagramme de Décision Binaires

b) Réduire l'arbre de décision en DDB

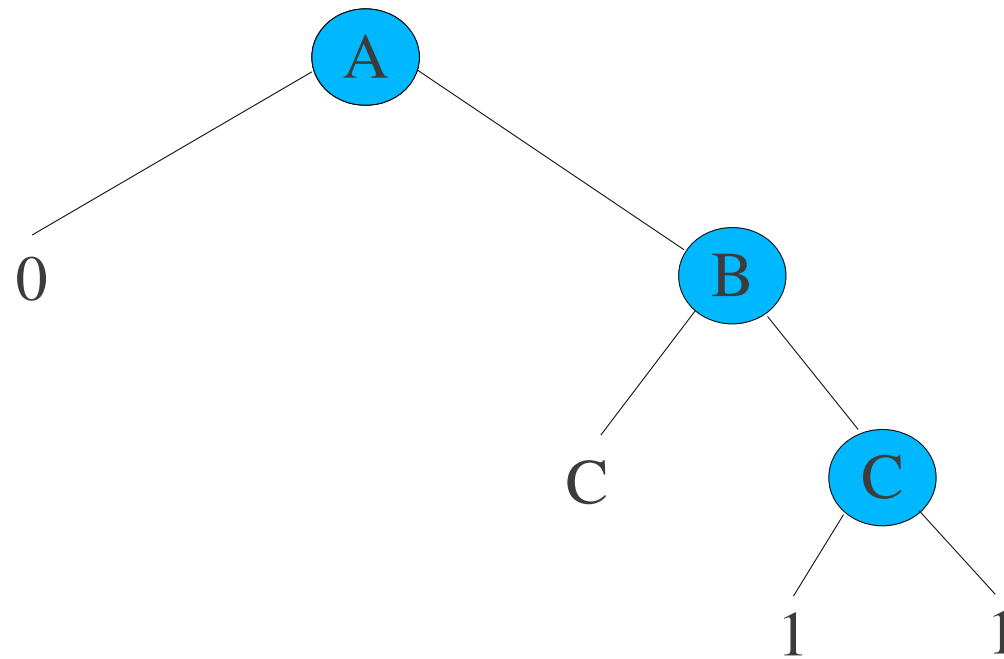
- De la gauche vers la droite, remplacer les feuilles par des constantes équivalentes ou des variables et élaguer les branches



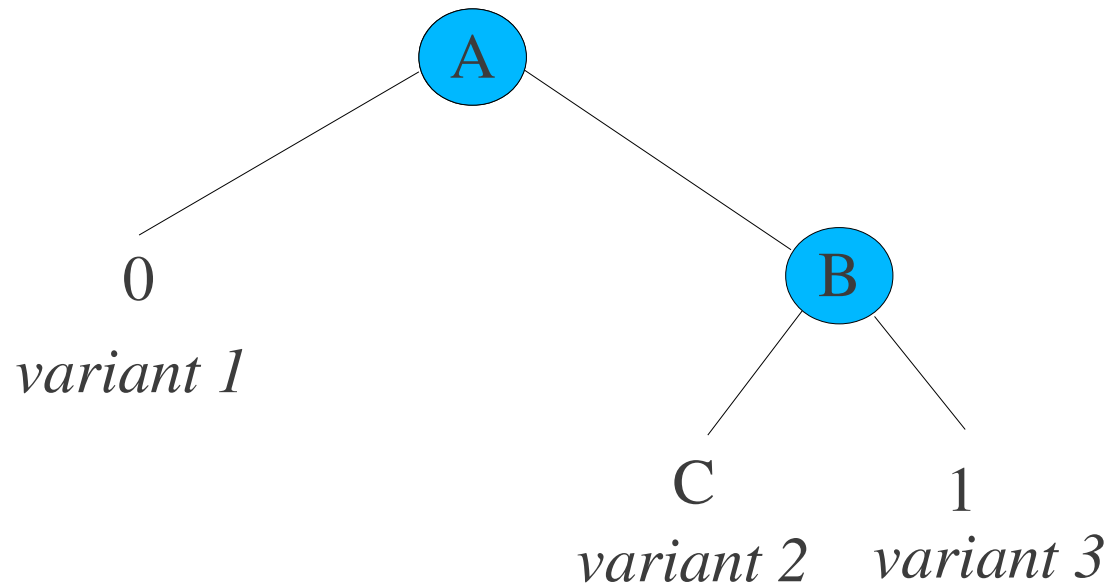
Tables de Décisions – Déterminants de Diagramme de Décision Binaires



Tables de Décisions – Déterminants de Diagramme de Décision Binaires



Tables de Décisions – Déterminants de Diagramme de Décision Binaires



Tables de Décisions – Déterminants de Diagramme de Décision Binaires

2. Transcrire le DDB en table de déterminants DDB

- table de déterminants DDB chemins de la racine aux feuilles du DDB

BDD Variant	A	B	C	Z
1	F	-	-	F
2	T	F	C	C
3	T	T	-	T

Tables de Décisions – Déterminants de Diagramme de Décision Binaires

3. Générer la suite de tests DDB

BDD Variant	A	B	C	Z
1	F	-	-	
2	T	F	F	
2	T	F	T	X
3	T	T	-	X

- Des valeurs doivent être assignées aux variables

Don't Care

- de façon aléatoire ou
- par suspicion

Tables de Décisions – Négation de Variables

- Stratégie pour générer des cas de tests permettant de trouver les défauts résultant de l'implémentation des Don't Care
 - La stratégie de déterminants de DDB ne traite pas les variables Don't Care
- La spécification doit être donnée sous forme de somme-de-produits de termes ((a11 *and* a12 ... *and* a1k) *or* (a21 *and* a22 ... *and* a2l) *or* ...)

Tables de Décisions – Négation de Variables

- Produit une suite de test incluant:
 - une variante pour chacun des termes produits, tels qu'aucun autre produit n'est *vrai* (*unique true points*)
 - une variante pour chacun des termes résultant de la négation de chacun des littéraux de chacun des termes produits, tels que la formule évalue à faux (*near false points*)

Tables de Décisions – Négation de Variables

- Exemple: $Z = A \text{ and } (B \text{ or } C) = (A \text{ and } B) \text{ or } (A \text{ and } C)$

0. Unique true points

- $(A \text{ and } B) \text{ true, } (A \text{ and } C) \text{ false } \{A=T, B=T, C=F\}$
- $(A \text{ and } B) \text{ false, } (A \text{ and } C) \text{ true } \{A=T, B=F, C=T\}$

Tables de Décisions – Négation de Variables

- Exemple: $Z = A \text{ and } (B \text{ or } C) = (A \text{ and } B) \text{ or } (A \text{ and } C)$

1. Near false points

– $(A \text{ and } B)$

- $(\sim A \text{ and } B) \{A=F, B=T, C=T\}$
- $(A \text{ and } \sim B) \{A=T, B=F, C=F\}$

– $(A \text{ and } C)$

- $(\sim A \text{ and } C) \{A=F, B=F, C=T\}$
- $(A \text{ and } \sim C) \{A=T, B=F, C=F\}$

Graphes Causes-effets

- Approche systématique d'aide à la sélection d'ensemble de cas de tests à haute efficacité
 - identification/analyse de relations d'une table de décision
 - génération de formule booléenne
- Noeuds:
 - Cause – instance distincte de condition donnée ou CE
 - Effet – résultat observable d'un changement de l'état du système
 - Noeud Intermédiaire - combinaison de causes représentant une expression sur des conditions de données

Graphes Causes-effets - layout



Causes

Noeuds Intermédiaires

Effets

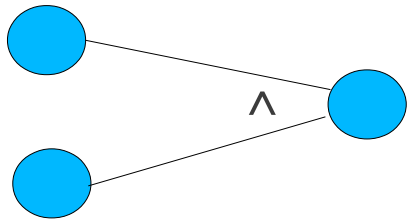
Liens valides de

- cause à noeud intermédiaire
- cause à effets
- noeuds intermédiaire à noeuds intermédiaire
- noeuds intermédiaire à effet

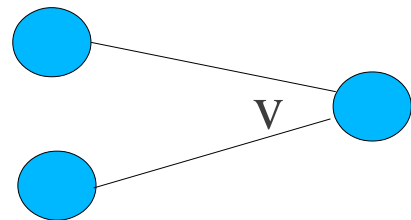
Graphes Causes-effets – types de liens



cause directe



AND



OR



NOT

Graphes Causes-effets - exemple

Une mise à jour de fichier dépend de la valeur dans deux champs. La valeur dans le champs 1 doit être ``A" ou ``B ". La valeur dans le champs 2 doit être un chiffre. Dans cette situation la mise à jour du fichier est faite. Si la valeur dans le champs 1 est incorrecte, le message d'erreur X12 est affiché. Si la valeur dans le champs 2 est incorrecte, le message d'erreur X13 est affiché.

Graphes Causes-effets - exemple

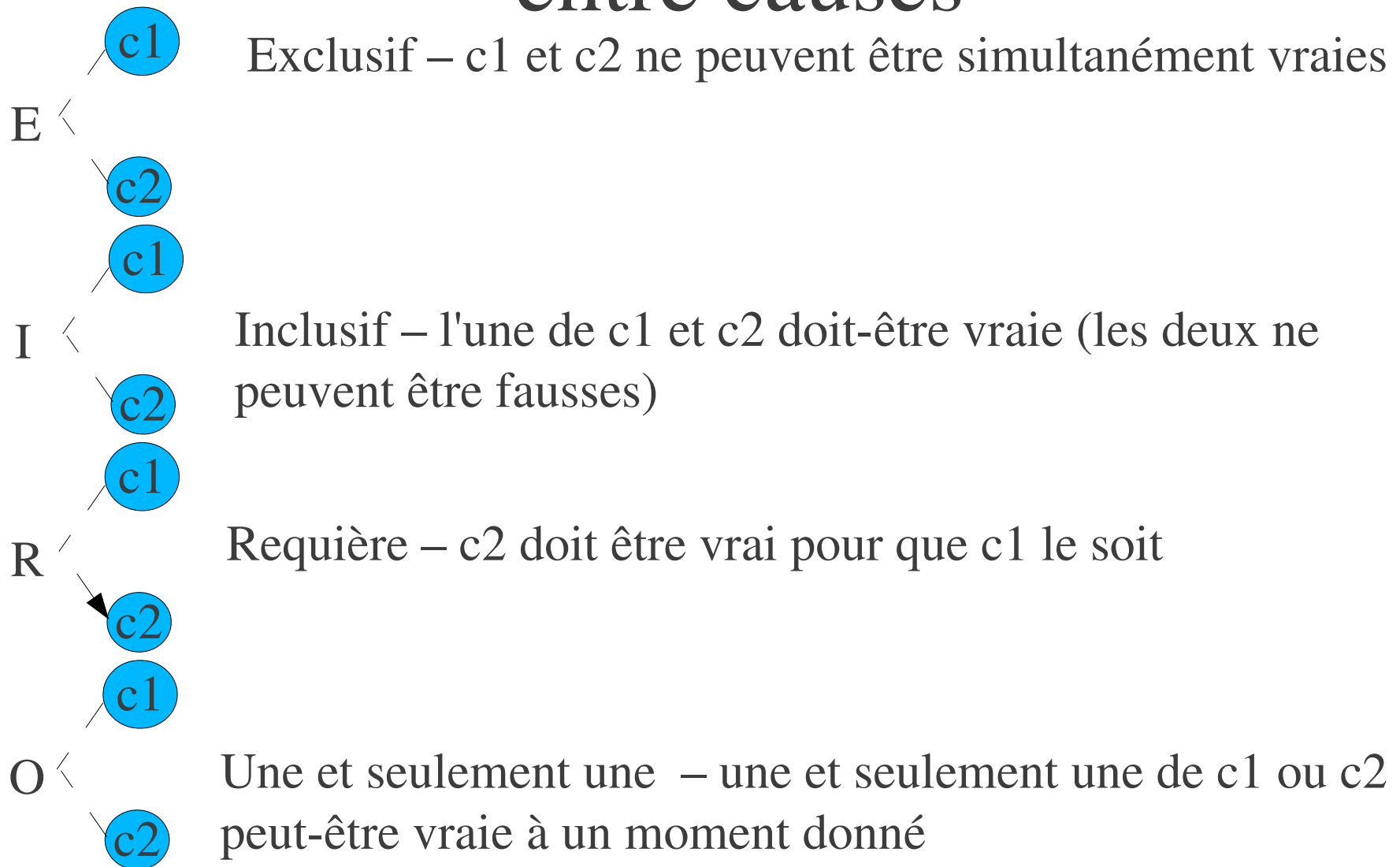
◆ Causes

- c1: caractère dans champs 1 est "A"
- c2: caractère dans champs 1 est "B"
- c3: caractère dans champs 2 est un digit

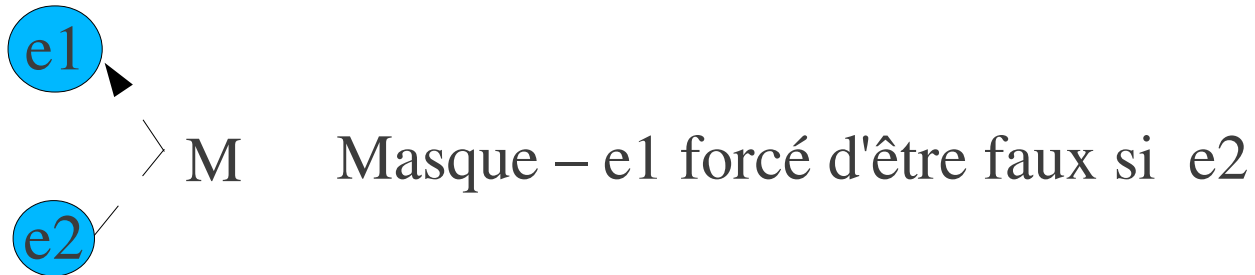
◆ Effets

- e1: mise à jour effectuée
- e2: message X12
- e3: message X13

Graphes Causes-effets – contraintes entre causes



Graphes Causes-effets – contraintes entre effets



Génération de cas de tests à partir de graphes causes-effets

- Décomposer la spécification en parties de taille raisonnable
- Définissez des graphes séparés par partie
 1. Identifier les Causes et Effets
 2. Déduisez les relations Logiques et Contraintes – représentez comme un graphe
 3. Développez une table de décision
 4. Traduisez chacune des variantes de la table de décision en cas de test.

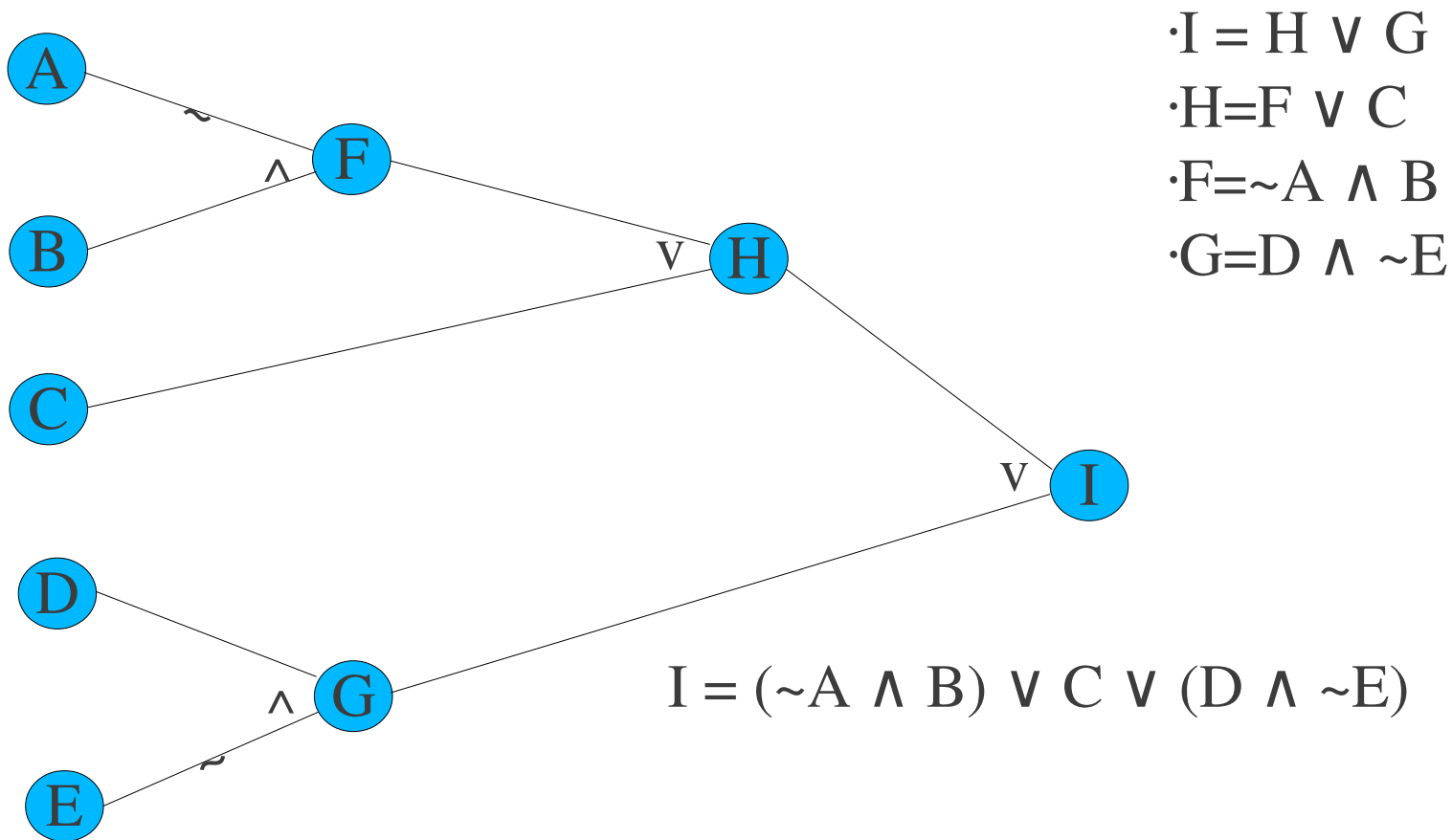
Génération de cas de tests à partir de graphes causes-effets

- Pour déterminer la combinaison de conditions uniques
- Travaillez avec un seul effet à la fois
 1. Mettez l'effet dans l'état *vrai* (1)
 2. Recherchez en allant à reculons dans le graphe toutes les combinaisons d'entrées forçant l'effet à l'état vrai
 - ♦ (nombre de combinaisons limité par contraintes)
 3. Créer une colonne dans la table de décision par combinaison
 4. Déterminez l'état de tous les autres effets pour chaque combinaison

Génération de Formule Booléenne à partir de graphes Cause-effets

- En allant des effets vers les causes
 1. Transcrivez les formules noeud-à-noeud à partir du graphe
 - écrire la formule correspondante à chaque effet ainsi que ses prédécesseurs
 - Pour chaque noeud intermédiaire
 - écrire la formule pour le noeud intermédiaire ainsi que ses prédécesseurs
 2. Dérivez la formule booléenne complète
 - remplacer les variables intermédiaires par substitution jusqu'à ce que la formule résultante ne comprenne que des causes
 - factoriser et re-écrire la formule sous forme de somme-de-produits

Génération de Formule Booléenne à partir de graphes Cause-effets



Dérivation de cas de tests d'exigences fonctionnelles

- Implique la clarification et reformulation des exigences
 - tel qu'elles soient testables
- But : Obtenir une formulation testable (sous forme point)
 - énumérer les exigences *uniques*
 - grouper les exigences liées

Dérivation de cas de tests d'exigences fonctionnelles

- Pour chaque exigence - développer
 - Un cas de test montrant le fonctionnement de l'exigence
 - Un cas de test essayant de la falsifier
 - ex: en essayant quelque-chose non permis
 - Tester les bornes et contraintes lorsque possible

Dérivation de cas de tests d'exigences fonctionnelles

- Exemple: Exigences d'un système de location de films
- 1. Le système permettra la location et retour de films
 - 1.1. Si un film est disponible pour location, il peut être prêté à un client.
 - 1.1.1 Un film est disponible pour la location jusqu'à ce que toutes les copies aient été simultanément empruntées.
 - 1.2. Si un film était non-disponible pour la location, alors le retour du film le rend disponible.
 - 1.3. La date de retour est établie quand un film est prêté et doit être montrée quand le film est retourné.
 - 1.4. Il doit être possible de déterminer l'emprunteur courant d'un film loué par une requête sur celui-ci.
 - 1.5. Une requête sur un membre indiquera tous les films que celui-ci à en location.

Dérivation de cas de tests d'exigences fonctionnelles

- Situations de tests pour l'exigence 1.1
 - Essayer d'emprunter un film disponible.
 - Essayer d'emprunter un film non-disponible.
- Situations de tests pour l'exigence 1.1.1
 - Essayer d'emprunter un film pour lequel il y a plusieurs copies, toutes empruntées.
 - Essayer d'emprunter un film pour lequel toutes les copies sauf une ont été empruntées.

Dérivation de cas de tests d'exigences fonctionnelles

- Situations de tests pour l'exigence 1.2
 - Emprunter un film non-disponible.
 - Retourner un film et l'emprunter
- Situations de tests pour l'exigence 1.3
 - Emprunter un film, le retourner et vérifier les dates.
 - Vérifier la date d'un film non-retourné.

Dérivation de cas de tests d'exigences fonctionnelles

- Situations de tests pour exigence 1.4
 - Requête sur un film emprunté.
 - Requête sur un film retourné.
 - Requête sur un film venant d'être retourné.
- Situations de tests pour exigence 1.5
 - Requête concernant un membre sans films.
 - Requête concernant un membre avec 1 film.
 - Requête concernant un membre avec plusieurs films.

Dérivation de Cas de Tests à partir de Cas d'utilisations

- Pour chacun des cas d'utilisations
 1. Développer un Graphe de Scénarios
 2. Déterminer tous les scénarios possibles
 3. Analyser et ordonner les scénarios
 4. Générer des cas de tests à partir des scénarios pour atteindre l'objectif de couverture
 5. Exécuter les cas de tests

Graphe de Scénarios

Généré d'un cas d'utilisation

- Noeuds correspondent à des points d'attente d'événements
 - de l'environnement, réaction système
- Il y a un *noeud de départ* unique
- Fin du cas d'utilisation est un *noeud de terminaison*
- Arcs correspondent à l'occurrence des événements
 - Peut inclure des *conditions*
 - Arc de boucle spécial
- Scénario:
 - *chemin* de noeud de départ à noeud de terminaison

Graphe de scénarios

Title: User login

Actors: User

Precondition: System is ON

1: User inserts a Card

2: System asks for Personal Identification Number (PIN)

3: User types PIN

4: System validates USER identification

5: System displays a welcome message to USER

6: System ejects Card

Alternatives:

1a: Card is not regular

1a1: System emits alarm

1a2: System ejects Card

4a: User identification is invalid
AND number of attempts < 4

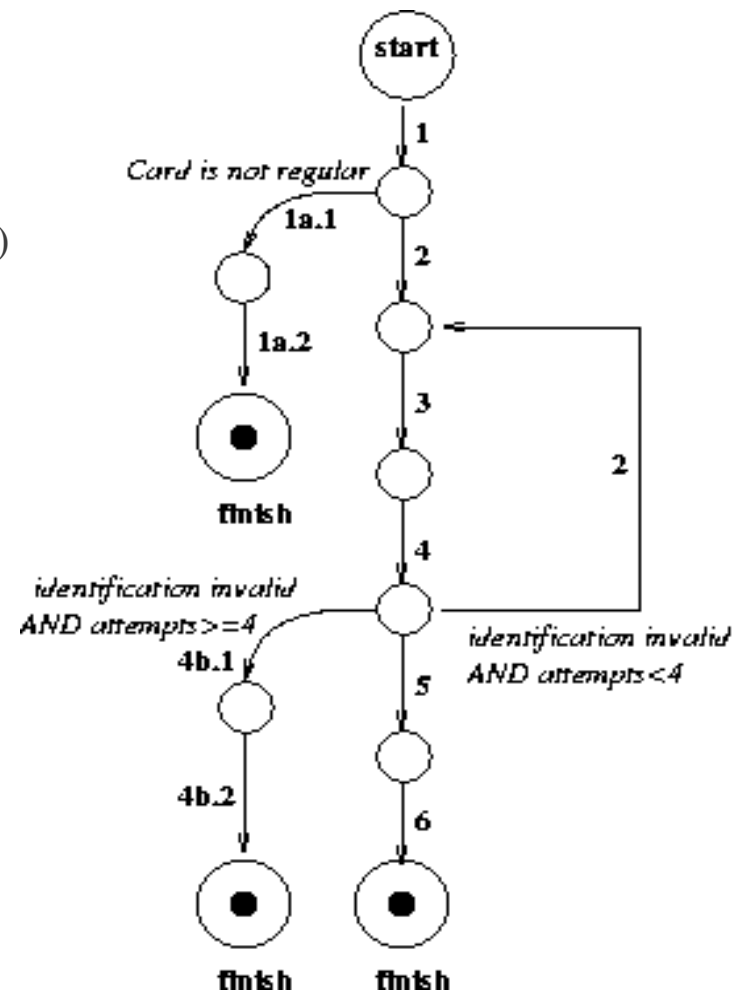
4a.1 Go back to Step 2

4b: User identification is invalid
AND number of attempts >= 4

4b.1: System emits alarm

4b.2: System ejects Card

Postcondition: User is logged in



Scénarios

- Chemin du début à la fin
- Boucles doivent être restreintes pour obtenir un nombre fini de scénarios

Id	Events	Description
1	1 - 2 - 3 - 4 - 5 - 6	User login with regular card, right PIN on first trial. Normal course of events
2	1 - 1a.1 - 1a.2	User login with non regular Card
3	1 - 2 - 3 - 4 - 2 - 3 - 4 - 5 - 6	User login with regular card, wrong PIN on first trial, the right PIN on second trial
4	1 - 2 - 3 - 4 - 2* - 3 - 4 - 4b.1 - 4b.2	User login with regular card, wrong PIN on first, second, third and fourth trials

Ordonnancement des Scénarios

- Si il y a trop de scénarios pour tout tester
- Ordonnancement peut-être basé sur criticalité et fréquence
- Inclure toujours le *scénario primaire*
 - devrait être testé en premier

Génération de Cas de Tests

- Selon l'objectif de couverture. Ex:
 - Toutes les branches du graphe de scénarios (objectif minimal)
 - Tous les Scénarios
 - n scénarios les plus critiques

Exemple de cas de Test

Cas de Test: TC1

Objectif: Test the main course of events for the ATM system.

Référence Scénario: 1

Setup: Create a Card #2411 with PIN #5555 as valid user identification, System is ON

Cours des événements

	External Event	Reaction	Comment
1	User insert Card #2411	System asks for Personal Identification Number (PIN)	
2	User types PIN #5555	System validates USER identification System displays a welcome message to USER	

Critère de succès: User is logged in

Dérivation de tests à partir de cas d'utilisations

- Nécessité parfois de combler les «lacunes»
 - Spécification incomplète de situations invalides
 - Validation avec l'équipe des exigences
- Toujours considérer les valeurs aux bornes